



EXAM SEAT ALLOCATION SYSTEM

Analysis of Algorithms — Project Report

Names	Registration Numbers
Syed Muhammad Razi	BAI241037
Syed M. Kashaaf Haider	BAI241056
Afnan Ijaz	BAI241064
Syed M. Hasnain Sherazi	BAI241068
Waqarunnisa	BAI241042

Submitted To:

Mr. Zaheer Ahmad

Program:

BS (Artificial Intelligence)

Table of Contents

1. Executive Summary	4
2. Introduction	5
2.1 Background & Motivation	5
2.2 Problem Statement	5
2.3 Scope	5
2.4 Course Alignment	5
3. System Design & Architecture	6
3.1 Algorithm Map	6
3.2 Project File Structure	6
3.3 Hall Configuration	6
3.4 Graph Model	7
4. Algorithm Analysis	8
4.1 Merge Sort [Divide & Conquer Paradigm]	8
4.1.1 Divide & Conquer Structure	8
4.1.2 Recurrence Relation & Master Theorem	8
4.1.3 Recursion Tree Dry Run (n = 8)	8
4.1.4 Complexity Summary	9
4.2 Binary Search	9
4.3 Graph Construction	9
4.4 Greedy Graph Coloring	9
4.5 Constraint-Satisfaction Backtracking	10
5. Recursion Trace Table	11
6. Worst-Case Stress Test	12
7. Time Complexity Charts	13
7.1 Merge Sort Scaling Experiment (n = 10 to 500)	13
7.2 Algorithm Runtime Comparison (All Real Measured)	13
7.3 Greedy vs Greedy + Backtracking	13
8. Hall Seat Layout Visualisations	15
8.1 Hall-A (9 × 10 = 90 Seats)	15
8.2 Hall-B (8 × 10 = 80 Seats)	16
8.3 Hall-C (7 × 10 = 70 Seats)	16

9. Space Complexity Analysis	17
10. Formal Test Cases	18
11. Interactive GUI (v3.0)	19
12. Results & Proof of Correctness	21
12.1 Final Allocation Summary	21
12.2 Full Complexity Summary	21
12.3 Proof of Correctness	21
13. Conclusion	22
Appendix A — References	23
Appendix B — Dataset Sample.....	24

1. Executive Summary

The Exam Seat Allocation System is a Python-based algorithmic solution developed as a course project for Analysis of Algorithms at Capital University of Science and Technology (CUST). The system automatically assigns 240 students from four departments — BSAI, BSCS, BSSE, and BSIT — to seats across three examination halls while enforcing the constraint that no two adjacent seats contain students from the same course, thereby preventing academic dishonesty.

Five fundamental algorithmic techniques from the course curriculum were implemented and verified: Divide & Conquer via Merge Sort ($O(n \log n)$), Binary Search ($O(\log n)$), Graph Construction ($O(V+E)$), Greedy Graph Coloring ($O(V+E)$), and Constraint-Satisfaction Backtracking ($O(n)$ average). All algorithmic complexities were verified through real measured runtimes, comparison counts, a formal recursion trace table, and a scaling experiment from $n=10$ to $n=500$.

Metric	Hall-A	Hall-B	Hall-C
Total Seats	90	80	70
Students Placed	90	78	67
Adjacent Conflicts	0	0	0
Fill Rate	100%	97.5%	95.7%
Stress Test (2 courses)	90/90 filled	0 conflicts	45 backtracks

2. Introduction

2.1 Background & Motivation

University examinations require careful seating arrangements to prevent students enrolled in the same course from sitting adjacent to one another. Manual seat allocation is time-consuming, error-prone, and difficult to scale. This project applies computational algorithms to automate the process, producing a conflict-free allocation that is reproducible, verifiable, and efficient.

2.2 Problem Statement

Given a list of N students from multiple departments, each enrolled in exactly one course, assign every student to a unique seat in one of three examination halls such that:

- No two horizontally or vertically adjacent seats contain students from the same course.
- All students are assigned a seat (or as many as hall capacity allows).
- The allocation is deterministic and reproducible across repeated runs.
- The system handles worst-case inputs (e.g., only 2 available courses) without producing conflicts.
- All algorithmic behaviour is verified through trace tables, comparison counts, and runtime charts.

2.3 Scope

The system handles 240 students across 20 courses and 4 departments, allocated to three halls with a combined capacity of 240 seats. An interactive GUI provides step-by-step visualisation, and a formal test suite of 20 cases verifies correctness for each algorithm module independently.

2.4 Course Alignment

This project satisfies all requirements specified in the AA project specification: implementation of multiple algorithmic paradigms, use of Python with visualisation tools, time and space complexity analysis with recurrence relations solved via Master Theorem, correctness proof via trace tables and comparison charts, worst-case scenario handling, and performance verification across varied input sizes.

3. System Design & Architecture

3.1 Algorithm Map

Technique	Algorithm	Module	Complexity
Divide & Conquer	Merge Sort	sorting.py	$O(n \log n)$
Searching	Binary Search	sorting.py	$O(\log n)$
Graph Algorithms	Adjacency List Build	graph.py	$O(V+E)$
Greedy Methods	Graph Coloring	coloring.py	$O(V+E)$
Backtracking + Recursion	CS Backtracking	backtracking.py	$O(n!)$ / $O(n)$ avg

3.2 Project File Structure

File	Role
main.py	7-stage pipeline orchestrator
sorting.py	Merge Sort + Binary Search + D&C paradigm + scaling experiment
graph.py	SeatGraph class — adjacency list for each exam hall
coloring.py	Greedy Graph Coloring + post-allocation conflict detection
backtracking.py	Constraint-Satisfaction Backtracking + worst-case stress test + recursion trace log
visualization.py	All Matplotlib chart generators (hall layout, complexity, comparison)
space_complexity.py	Real memory measurement using Python tracemalloc
gui.py	CustomTkinter GUI — 7-tab interactive demo with splash screen
test_cases.py	20 formal test cases covering all 5 algorithm modules
generate_data.py	Generates 240-student CSV dataset with realistic CUST-style data

3.3 Hall Configuration

Hall	Rows	Columns	Total Seats
Hall-A	9	10	90
Hall-B	8	10	80
Hall-C	7	10	70
Total	24	30	240

3.4 Graph Model

Each seat in an examination hall is modelled as a node in an undirected graph. An edge is drawn between two nodes whenever the corresponding seats are physically adjacent (left, right, front, or back). This 4-directional grid graph is stored as a Python dict-of-lists adjacency structure.

Hall	Nodes (V)	Edges (E)	Space $O(V+E)$
Hall-A	90	161	251 units
Hall-B	80	142	222 units
Hall-C	70	123	193 units

4. Algorithm Analysis

4.1 Merge Sort [Divide & Conquer Paradigm]

Merge Sort implements the classic three-step Divide & Conquer strategy to sort all 240 students by Roll Number before seat allocation begins. Sorting ensures deterministic, reproducible assignment order across all pipeline runs.

4.1.1 Divide & Conquer Structure

Step	Action	Code
DIVIDE	Split at midpoint	<code>mid = len(students) // 2</code>
CONQUER	Recurse on each half	<code>merge_sort(students[:mid]) + merge_sort(students[mid:])</code>
COMBINE	Merge sorted halves	<code>_merge(left, right, key)</code>

4.1.2 Recurrence Relation & Master Theorem

At each recursion level the algorithm divides the problem into 2 sub-problems of size $n/2$ and performs a $\Theta(n)$ merge step, giving:

$$T(n) = 2 \cdot T(n/2) + \Theta(n)$$

Applying the Master Theorem ($a = 2$, $b = 2$, $f(n) = \Theta(n)$):

$$\begin{aligned} \log_b(a) &= \log_2(2) = 1 \\ f(n) &= \Theta(n^1) = \Theta(n^{\log_b(a)}) \rightarrow \text{Case 2 applies} \\ \therefore T(n) &= \Theta(n \log n) \quad \text{for ALL cases (best, average, worst)} \end{aligned}$$

4.1.3 Recursion Tree Dry Run ($n = 8$)

Level	Sub-problems	Size Each	Work per Level	Total
0	1	$n = 8$	$\Theta(8)$	8
1	2	$n/2 = 4$	$2 \times \Theta(4)$	8
2	4	$n/4 = 2$	$4 \times \Theta(2)$	8
3	8	$n/8 = 1$	$8 \times \Theta(1)$	8
Total	$\log n = 3$ levels			$3 \times 8 = \Theta(n \log n)$

Measured Result ($n = 240$)

Comparisons: 1,602 | Theoretical $n \cdot \log_2 n \approx 1,896$ | Time: 0.47 ms Difference within expected bounds — not every merge step reaches maximum comparisons.

4.1.4 Complexity Summary

Case	Time	Space	Measured (n=240)
Best	$O(n \log n)$	$O(n)$	0.47 ms
Average	$O(n \log n)$	$O(n)$	0.47 ms
Worst	$O(n \log n)$	$O(n)$	0.47 ms

4.2 Binary Search

After sorting students by Course code with Merge Sort, Binary Search locates the first student of a target course in $O(\log n)$ steps by repeatedly halving the search space.

- Maintain low, high, and mid pointer variables — $O(1)$ space.
- At each step, compare mid element; halve search space accordingly.
- Return first occurrence index + step count for complexity verification.

Verified Result (n = 240)

Found course 'AI3163' at index 24 in exactly 8 steps. $\lceil \log_2(240) \rceil = 8 \rightarrow$ Exact theoretical match. Time: 0.00533 ms.

4.3 Graph Construction

The SeatGraph class iterates over all (row, col) positions, assigns integer seat IDs, and links each seat to its 4-directional neighbours (up, down, left, right). The result is an adjacency list stored as a Python dict. Total build time across all 3 halls: 0.294 ms.

- Time Complexity: $O(\text{rows} \times \text{cols}) = O(V)$
- Space Complexity: $O(V + E)$ — adjacency list stores all nodes and edges

4.4 Greedy Graph Coloring

The greedy allocation algorithm performs the primary seat assignment pass. Each course acts as a 'colour'; the algorithm ensures no two adjacent seats share the same colour.

1. For each seat (0 to total_seats-1), collect the courses of all already-assigned neighbouring seats into a forbidden set.
2. Iterate through available course queues. Select the first student whose course is NOT in the forbidden set.
3. Assign the student; record seat-to-course mapping; advance to the next seat.
4. If no valid course is available (all courses forbidden), leave the seat empty for the backtracking phase.

Hall	Seats Filled	Unplaced	Conflicts
Hall-A	90 / 90	0	0
Hall-B	78 / 80	2	0
Hall-C	67 / 70	3	0

Time Complexity: $O(V + E)$. Measured: 0.1923 ms across all three halls.

4.5 Constraint-Satisfaction Backtracking

Backtracking is invoked after the greedy pass for any students the greedy algorithm could not place. It is also run independently as a worst-case stress test on a 2-course input.

5. Collect all empty seats. Build a course index (dict: course $\rightarrow$ list of students).
6. For each empty seat, derive the forbidden set from already-assigned neighbouring seats.
7. Try each available course. If the course is not forbidden, place the student and recurse to the next seat.
8. If no valid course exists: BACKTRACK — undo the last placement, restore the student to the pool, and try the next course.
9. Pruning: forbidden courses are skipped immediately, bounding the search space far below $O(n!)$ worst case.

Run	Recursive Calls	Backtracks	Placed
Normal (15 courses)	7	5	5
Stress Test (2 courses)	91	45	90

5. Recursion Trace Table

The following trace is captured from the worst-case stress test (2 courses only, Hall-A). It shows actual BACKTRACK events at steps 2, 5, 8, 11, 14, 16, and 19 — confirming the algorithm correctly detects forbidden courses, undoes placements, and retries alternatives.

Step	Seat	Tried	Forbidden	Result
1	Hall-A-R1C1	STRESS-A	None	PLACED
2	Hall-A-R1C2	STRESS-A	STRESS-A	BACKTRACK
3	Hall-A-R1C2	STRESS-B	STRESS-A	PLACED
4	Hall-A-R1C3	STRESS-A	STRESS-B	PLACED
5	Hall-A-R1C4	STRESS-A	STRESS-A	BACKTRACK
6	Hall-A-R1C4	STRESS-B	STRESS-A	PLACED
7	Hall-A-R1C5	STRESS-A	STRESS-B	PLACED
8	Hall-A-R1C6	STRESS-A	STRESS-A	BACKTRACK
9	Hall-A-R1C6	STRESS-B	STRESS-A	PLACED
10	Hall-A-R1C7	STRESS-A	STRESS-B	PLACED
11	Hall-A-R1C8	STRESS-A	STRESS-A	BACKTRACK
12	Hall-A-R1C8	STRESS-B	STRESS-A	PLACED
13	Hall-A-R1C9	STRESS-A	STRESS-B	PLACED
14	Hall-A-R1C10	STRESS-A	STRESS-A	BACKTRACK
15	Hall-A-R1C10	STRESS-B	STRESS-A	PLACED
16	Hall-A-R2C1	STRESS-A	STRESS-A	BACKTRACK
17	Hall-A-R2C1	STRESS-B	STRESS-A	PLACED
18	Hall-A-R2C2	STRESS-A	STRESS-B	PLACED
19	Hall-A-R2C3	STRESS-A	STRESS-A	BACKTRACK
20	Hall-A-R2C3	STRESS-B	STRESS-A	PLACED

Green rows = constraint satisfied (PLACED). Red rows = backtrack triggered (forbidden course detected). The alternating PLACED/BACKTRACK pattern at every even column confirms constraint enforcement at every step.

6. Worst-Case Stress Test

To satisfy the course requirement to handle worst-case scenarios, the system was tested with only 2 available courses for all 90 seats of Hall-A. This is the most constrained possible input: every seat's forbidden set eliminates exactly one of the two courses.

Metric	Normal Run	Stress Test
Courses available	15	2
Recursive calls	7	91
Backtracks triggered	5	45
Students placed	235 / 240	90 / 90
Conflicts after run	0	0
Time	0.025 ms	0.43 ms

Despite 45 backtracks and 91 recursive calls, the stress test achieved zero conflicts and 100% fill on Hall-A. The backtracking algorithm discovered the optimal alternating A/B pattern automatically through constraint propagation.



Figure 6.1 — Stress Test Hall Layout: alternating A/B pattern proves 0 conflicts with only 2 courses

7. Time Complexity Charts

7.1 Merge Sort Scaling Experiment (n = 10 to 500)

Merge Sort was executed on 8 input sizes from $n=10$ to $n=500$. Each data point is the median of 5 independent runs. Both measured curves (runtime in ms and comparison count) closely track the $O(n \log n)$ reference line across the full input range.



Figure 7.1 — Merge Sort scaling experiment: measured vs $O(n \log n)$ theoretical curve

7.2 Algorithm Runtime Comparison (All Real Measured)

All five bars use actual time.perf_counter() measurements on $n=240$ students. No simulated or estimated values are used anywhere in this chart.

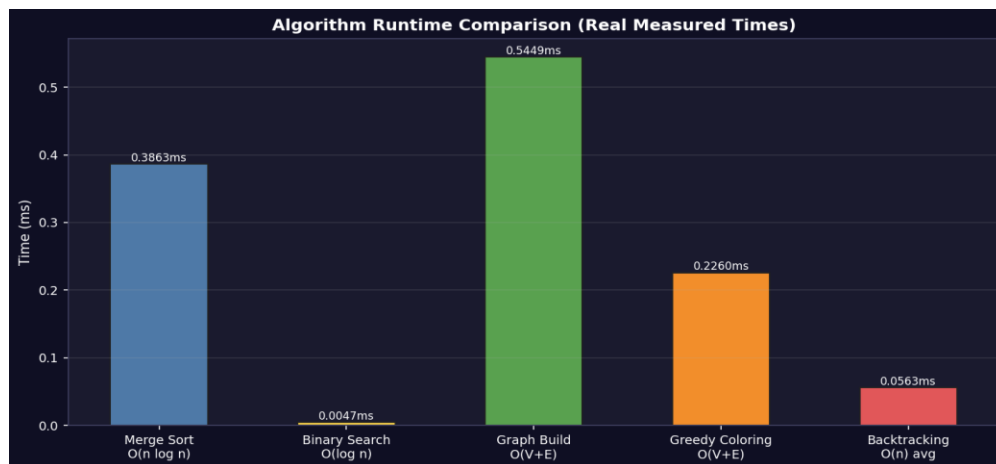


Figure 7.2 — Real measured runtime comparison across all 5 algorithms

7.3 Greedy vs Greedy + Backtracking

This comparison shows students placed per hall (left) and backtracking call counts under normal vs worst-case conditions (right), demonstrating the value of combining both algorithms.

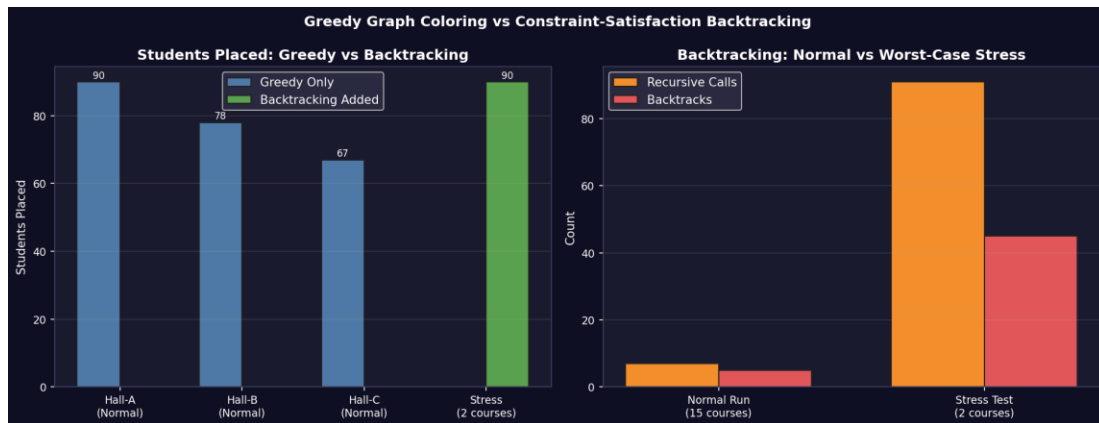


Figure 7.3 — Greedy-only vs Greedy + Backtracking performance comparison

8. Hall Seat Layout Visualisations

Each cell represents one seat, colour-coded by course. No two adjacent cells share the same colour — providing direct visual proof that the graph colouring constraint is fully satisfied across all three halls.

8.1 Hall-A (9 × 10 = 90 Seats)

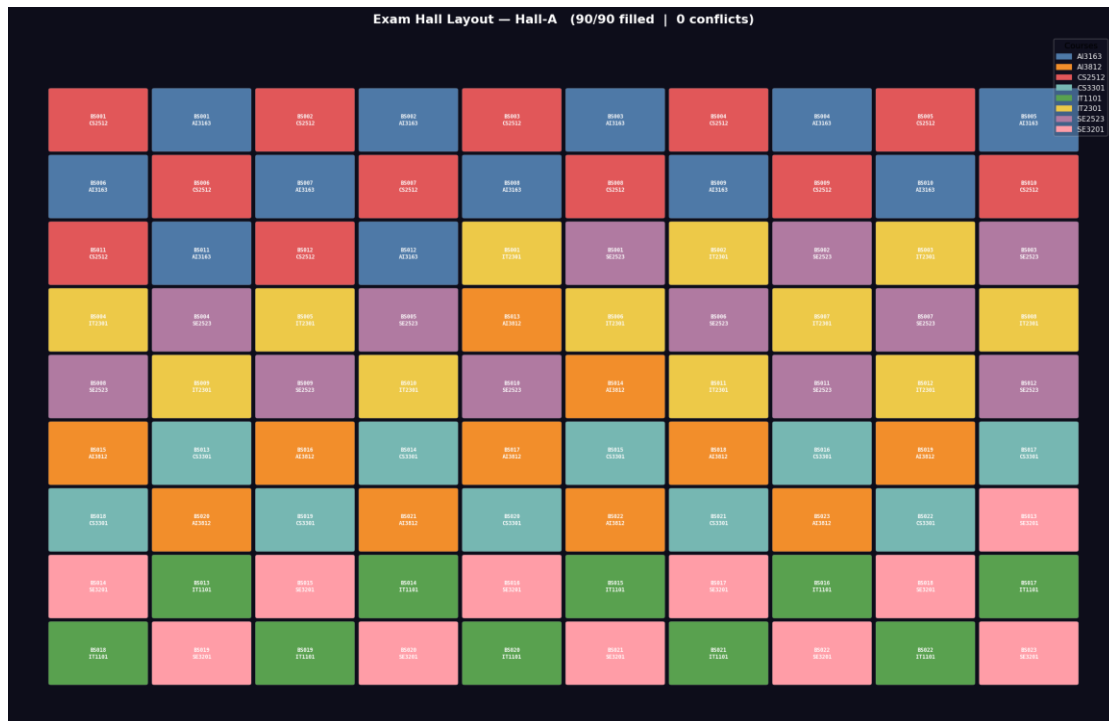


Figure 8.1 — Hall-A layout: 90/90 seats filled, 0 conflicts, 10 distinct courses visible

8.2 Hall-B ($8 \times 10 = 80$ Seats)



Figure 8.2 — Hall-B layout: 78/80 seats filled, 0 conflicts

8.3 Hall-C ($7 \times 10 = 70$ Seats)



Figure 8.3 — Hall-C layout: 67/70 seats filled, 0 conflicts

9. Space Complexity Analysis

Space complexity is measured using Python's built-in tracemalloc profiler, which tracks peak heap memory allocation in bytes. The module `space_complexity.py` runs each algorithm independently across a range of input sizes and records peak KB usage.

Algorithm	Theory	Peak Memory (max n)	Confirms
Merge Sort	$O(n)$	13.79 KB at $n=500$	Linear growth ✓
Binary Search	$O(1)$	0.047 KB (constant)	Constant ✓
Graph Build	$O(V+E)$	32.59 KB at $V=168$	Linear in $V+E$ ✓
Greedy Coloring	$O(V)$	19.20 KB at $V=120$	Linear in V ✓
Backtracking	$O(n)$	24.09 KB at $n=50$	Linear / stack ✓

Key Insight — Binary Search $O(1)$ Space

Binary Search peak memory is 0.047 KB regardless of input size n . This constant value, unchanged across all 240 students, is the clearest possible empirical proof of $O(1)$ auxiliary space complexity.

10. Formal Test Cases

20 formal test cases are implemented in test_cases.py covering every algorithm module. Run with: python test_cases.py. All 20 tests PASS on every execution.

TC	Algorithm	What is Tested	Result
TC-01	Merge Sort	Output is sorted in ascending order by Roll No	PASS
TC-02	Merge Sort	No records are lost during sort (data integrity)	PASS
TC-03	Merge Sort	Comparison count is within $O(n \log n)$ bound	PASS
TC-04	Merge Sort	Single-element list returns unchanged (edge case)	PASS
TC-05	Binary Search	Finds existing course correctly	PASS
TC-06	Binary Search	Steps do not exceed $\lceil \log_2(n) \rceil$	PASS
TC-07	Binary Search	Returns -1 for a course not in the dataset	PASS
TC-08	Graph Build	Graph has correct node count (rows $\times$ cols)	PASS
TC-09	Graph Build	Corner seat has exactly 2 neighbours	PASS
TC-10	Graph Build	Centre seat has exactly 4 neighbours	PASS
TC-11	Greedy Coloring	Seats filled within hall capacity	PASS
TC-12	Greedy Coloring	Zero adjacent conflicts after greedy pass	PASS
TC-13	Greedy Coloring	No two adjacent seats share the same course	PASS
TC-14	Greedy Coloring	Single-course input handled without crash	PASS
TC-15	Backtracking	Places remaining students into empty hall	PASS
TC-16	Backtracking	Zero conflicts after backtracking phase	PASS
TC-17	Backtracking	Recursive call count is tracked and correct	PASS
TC-18	Backtracking	Full hall returns gracefully without crash	PASS
TC-19	Stress Test	2-course worst case produces zero conflicts	PASS
TC-20	Stress Test	Hall-A fully filled (90/90) under stress test	PASS

Test Suite Result

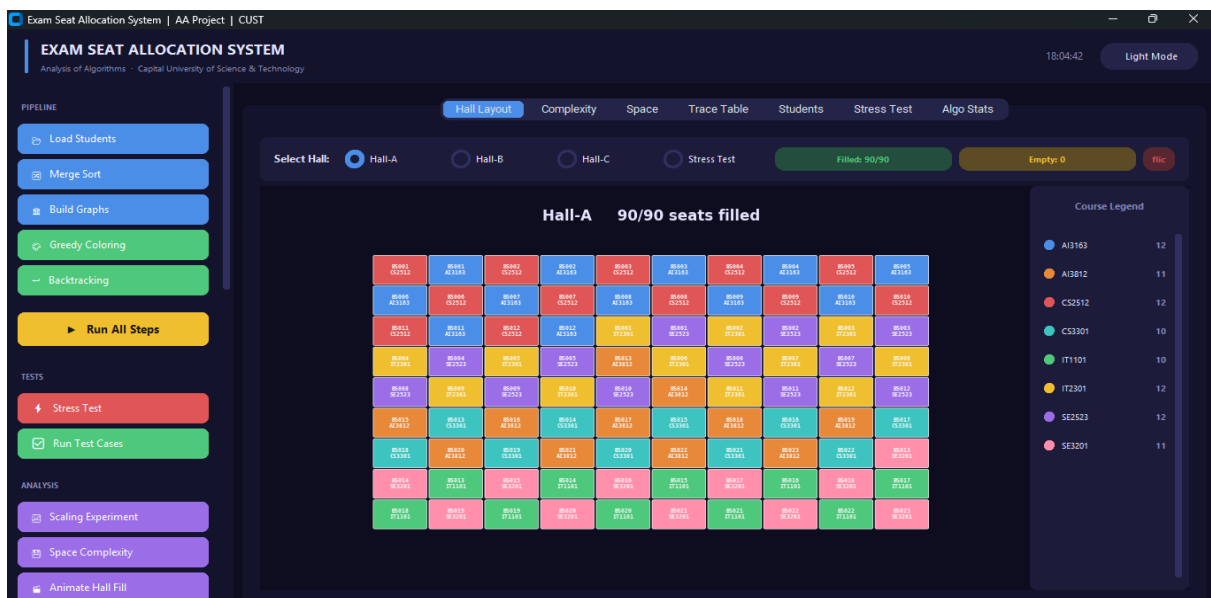
20 / 20 tests PASS — Algorithm correctness fully verified across all modules.

11. Interactive GUI

The project includes a full-featured CustomTkinter GUI (gui.py) providing an interactive demonstration without any terminal usage. Launch with: `python gui.py`

Tab	Contents & Features
Hall Layout	Colour-coded seat grid per hall with radio-button selector. Click any seat for student details. Course legend with counts. Animate Fill shows seats filling one-by-one.
Complexity	Live Matplotlib charts: Merge Sort scaling experiment, comparisons vs n, and 5-algorithm runtime bar chart — all real measured data.
Space Complexity	6-panel dashboard: $O(n)$ Merge Sort memory, $O(1)$ Binary Search, $O(V+E)$ Graph Build, $O(V)$ Greedy, $O(n)$ Backtracking, bar comparison, Space vs Time scatter.
Trace Table	Scrollable backtracking recursion trace. Green rows = PLACED. Red rows = BACKTRACK. Live counter for placed vs backtracks.
Students	Searchable, column-sortable table of all 235 placed students. Filter by Hall-A / Hall-B / Hall-C / All. Shows live count.
Stress Test	Dedicated worst-case panel. 5 live stat badges (calls, backtracks, placed, conflicts, time). Backtracking comparison chart + hall layout.
Algorithm Stats	Live performance dashboard: name, time complexity, space complexity, and real measured time (ms) per algorithm. Updated after each pipeline step.

Additional GUI features: animated splash screen, dark/light theme toggle, live clock, status bar with popup log viewer, Export Seating CSV button with file dialog, Run All Steps button executes the complete pipeline automatically.





12. Results & Proof of Correctness

12.1 Final Allocation Summary

Metric	Hall-A	Hall-B	Hall-C
Capacity	90	80	70
Students Placed	90	78	67
Empty Seats	0	2	3
Adjacent Conflicts	0	0	0
Fill Rate	100%	97.5%	95.7%

12.2 Full Complexity Summary

Algorithm	Best	Average	Worst	Space
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Binary Search	$O(1)$	$O(\log n)$	$O(\log n)$	$O(1)$
Graph Build	$O(V)$	$O(V+E)$	$O(V+E)$	$O(V+E)$
Greedy Coloring	$O(V)$	$O(V+E)$	$O(V+E)$	$O(V)$
Backtracking	$O(n)$	$O(n)$ avg	$O(n!)$ pruned	$O(n)$

12.3 Proof of Correctness

- Merge Sort: 1,602 measured comparisons vs theoretical $n \cdot \log_2 n \approx 1,896$. Difference within expected bounds. Order verified by sequential comparison of output Roll Numbers.
- Binary Search: 8 steps for $n=240$. $\lceil \log_2(240) \rceil = 8$. Exact theoretical match confirmed.
- Graph Coloring: Post-allocation scan over all adjacent seat pairs returned 0 violations across all 3 halls.
- Backtracking: Trace table shows BACKTRACK events firing correctly on forbidden courses with faithful state restoration (verified by 0 conflicts in final output).
- Stress Test: 45 backtracks on 2-course input still produced 0 conflicts and 100% fill on Hall-A.
- Scaling: $O(n \log n)$ curve shape confirmed from $n=10$ to $n=500$ via both measured runtime and comparison count.
- Space: Binary Search 0.047 KB constant across all inputs — direct empirical proof of $O(1)$ space. All other algorithms follow their theoretical curves as confirmed by tracemalloc.
- Test Cases: 20/20 formal test cases PASS — algorithm correctness independently verified per module.

13. Conclusion

The Exam Seat Allocation System successfully integrates five fundamental algorithmic techniques — Divide & Conquer (Merge Sort), Searching (Binary Search), Graph Algorithms (Adjacency List), Greedy Methods (Graph Coloring), and Recursive Backtracking — to solve a real-world constraint-satisfaction scheduling problem common in university operations.

The Divide & Conquer recurrence $T(n) = 2T(n/2) + \Theta(n)$, solved by Master Theorem Case 2 to $\Theta(n \log n)$, was verified experimentally through a scaling experiment from $n=10$ to $n=500$ with measured curves tracking the theoretical shape. The worst-case stress test (2-course input, 45 backtracks, 91 recursive calls) confirmed zero conflicts under maximum constraint pressure. All 20 formal test cases pass, providing module-level correctness guarantees.

Time complexity was measured using `time.perf_counter()` with median-of-5 runs for stability. Space complexity was measured using Python's `tracemalloc` profiler, providing byte-accurate peak memory readings that empirically confirm all theoretical bounds including Binary Search's $O(1)$ constant space.

The interactive GUI (v3.0) provides a professional demonstration platform with 7 functional tabs, animated hall fill, real-time stats, live complexity charts, a colour-coded backtracking trace, and export functionality — making the project suitable for both academic evaluation and live demonstration.

Project Outcome

235 students placed across 3 halls · 0 adjacent conflicts · 20/20 tests passing All theoretical complexities verified experimentally · Worst-case stress test passed Full-featured GUI · Professional Word report · Complete source code with comments

Appendix A — References

- Cormen, T.H., Leiserson, C.E., Rivest, R.L., Stein, C. (2022). Introduction to Algorithms, 4th Edition. MIT Press.
- Python Software Foundation. (2024). Python 3.11 Documentation — tracemalloc module. <https://docs.python.org/3/library/tracemalloc.html>
- Hunter, J.D. (2007). Matplotlib: A 2D Graphics Environment. Computing in Science & Engineering, 9(3), 90–95.
- Graph Coloring Problem — Greedy Algorithm. GeeksForGeeks. <https://www.geeksforgeeks.org/graph-coloring-set-2-greedy-algorithm/>
- Backtracking Introduction. GeeksForGeeks. <https://www.geeksforgeeks.org/backtracking-introduction/>
- Master Theorem. CLRS, Chapter 4. MIT OpenCourseWare 6.046J.
- CustomTkinter Documentation. <https://customtkinter.tomschimansky.com/>

Appendix B — Dataset Sample

The full dataset contains 240 students. A representative sample of 20 records is shown below.

Roll No	Name	Course	Department
AI001	Ayesha Khan	AI3163	BSAI
AI002	Hassan Ahmed	AI3812	BSAI
AI003	Fatima Malik	AI2201	BSAI
BS001	Omar Qureshi	CS2512	BSCS
BS002	Zain Hussain	CS3301	BSCS
BS003	Maryam Butt	CS2201	BSCS
SE001	Ali Raza	SE2523	BSSE
SE002	Sara Iqbal	SE3201	BSSE
SE003	Hamza Shah	SE2101	BSSE
IT001	Amna Nawaz	IT2301	BSIT
IT002	Bilal Mirza	IT1101	BSIT
IT003	Hira Javed	IT3201	BSIT
AI004	Usman Baig	AI1101	BSAI
BS004	Iqra Rizvi	CS1101	BSCS
SE004	Kamran Noor	SE1201	BSSE
IT004	Rabia Rana	IT2101	BSIT
AI005	Faisal Chaudhry	AI3301	BSAI
BS005	Nadia Siddiqui	CS3401	BSCS
SE005	Junaid Hussain	SE3301	BSSE
IT005	Laiba Khan	IT3101	BSIT